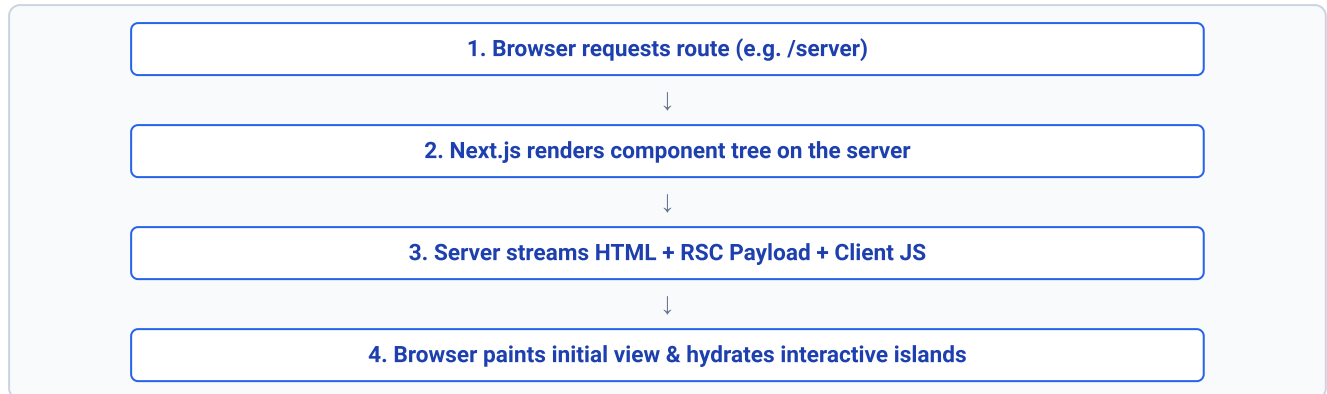


Server-Side Rendering with Next.js

Demonstrating Server and Client rendering via the Next.js App Router, React Server Components (RSC), and TypeScript.

WHAT IS SERVER-SIDE RENDERING?

Server-Side Rendering (SSR) means the server prepares the initial HTML for a route before transmitting it to the browser. The browser displays meaningful UI earlier, while JavaScript subsequently hydrates interactive client boundaries.



Key Terminology

- **Server Components:** Components executed exclusively on the server by default in App Router.
- **Static Generation (SSG):** Pre-rendering HTML ahead of requests during build time or via caching.
- **Hydration:** Attaching interactive React listeners/state to server-rendered static HTML.

WHY USE SSR?

SSR delivers populated DOM nodes on first response rather than returning an empty root div:

Key Advantages

- **SEO Optimization:** Search bots crawl fully populated page content instantly.
- **Fast First Paint:** Immediate visual feedback before JavaScript loads.
- **Secure Execution:** Database queries and secrets remain strictly server-side.

Trade-offs & Overhead

- **Server Compute:** Each dynamic request consumes CPU cycles.
- **Latency:** Dynamic data fetching delays initial byte delivery (TTFB).
- **Hydration Costs:** Client bundle needed for interactive widgets.

PROJECT STRUCTURE & SETUP

Dependencies (Next.js 16 / React 19):

```
{
  "dependencies": {
    "next": "16.3.5",
    "react": "19.2.8",
    "react-dom": "19.2.8"
  }
}
```

Directory Layout:

```
app/
  server/page.tsx # SSR Example
  client/page.tsx # CSR Example
  layout.tsx      # Root Layout
ClientComponent.tsx # "use client"
ServerComponent.tsx # Server boundary
```

STEP 2: CREATING A SERVER COMPONENT

In the App Router, files inside `app/` are Server Components by default unless marked otherwise:

```
import ClientComponent from "@/ClientComponent";
import ServerComponent from "@/ServerComponent";

export default async function ServerPage() {
  console.log("Server Component Execution!");

  return (
```

Server Side Rendering!

```
);
}
```

Server Component Rules & Capabilities

- `console.log` outputs directly to the Node.js server terminal, not the browser dev tools.
- Components can be native `async` functions and perform direct asynchronous work.
- Cannot access browser APIs (e.g., `window`, `localStorage`) or hooks (`useState`, `useEffect`).

STEP 3: FETCHING DATA ON THE SERVER

Server Components perform data fetching directly inside the render loop before HTML emission:

```
async function getData() {
  const response = await fetch("https://api.example.com/data");
  if (!response.ok) throw new Error("Failed to load data");
  return response.json();
}

export default async function ServerPage() {
  const data = await getData();
  return

  {JSON.stringify(data, null, 2)}

;
}
```

Dynamic Rendering vs. Caching Strategies

By default, Next.js caches routes when possible. Force request-time SSR using route segment configs:

STEP 4 & 5: CLIENT COMPONENT COMPOSITION

Declare interactive client boundaries using the `"use client"` directive at the top of the file:

```
"use client";

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    

setCount((val) => val + 1)>
      Count: {count}


  );
}
```

Passing Server Components as Children

Keep client boundaries low in the component tree. Pass Server Components as `children` to avoid turning static trees into client JavaScript:

```
// ClientComponent.tsx
"use client";

export default function ClientComponent({ children }: { children: React.ReactNode }) {
  return

  {children}

;
}
```

Serialization Requirement

Props passed across the Server-to-Client boundary **must be serializable** (e.g. strings, numbers, plain objects, booleans, arrays). Class instances, functions, and database handles cannot be passed to Client Components.

STEP 6: ROOT LAYOUTS & METADATA

Layouts wrap routes and execute on the server to inject global document headers without client JavaScript:

```
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "Server Rendering Demo",
  description: "A Next.js Server-Side Rendering example",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    

{children}


  );
}
```

SSR VS. CLIENT-SIDE RENDERING (CSR) CODE COMPARISON

Server-Side Rendering (SSR)

```
export default async function Page() {
  // Executed on server before sending HTML
  const res = await
  fetch("https://api/data");
  const data = await res.json();

  return
  {data.title}
;
}
```

Client-Side Rendering (CSR)

```
"use client";
import { useEffect, useState } from "react";

export default function Page() {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch("/api/data")
      .then(r => r.json())
      .then(setData);
  }, []);

  return
  {data?.title ?? "Loading..."}
;
}
```

COMMON PITFALLS TO AVOID

- Adding "use client" at the page level unnecessarily.
- Attempting to use hooks (useState) in Server Components.
- Exposing private server credentials inside client bundles.

- Expecting server console.log output in the browser console.
- Fetching initial render data inside client useEffect hooks.
- Forcing force-dynamic on purely static content.

WHEN NOT TO USE SSR

Opt for Client Components or Client-side SPA architectures when:

- Building highly interactive, authenticated dashboards with continuous state changes.
- UI relies exclusively on browser APIs (Canvas, WebGL, WebSocket handlers).
- SEO and initial loading speed are not primary requirements.

USEFUL CLI COMMANDS

```
npm run dev      # Starts local development server with Fast Refresh
npm run build    # Compiles and optimizes production build
npm start        # Serves compiled production build
npm run lint     # Executes ESLint checks
```